Review notes · for you

Algorithms and software engineering: unit check

Programming · Level 4 · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- How work grows: big O
- Recursion
- Data structures
- Testing like an engineer

Activity: Code review circle

1. Warm-up (3 min): Write one idea from each of the four lessons in your own words.
2. Write (7 min): Write a small function from this unit (first_repeat, balanced, digit_sum or search) with five asserts, including edge cases.
3. Review (7 min): Pass your code to the next person. They add one test they think will break it, and one helpful comment.
4. Fix (3 min): Get your code back and make every test pass, the new one included.

Discussion

- Which idea from this unit will you use in your next project?
- Which question surprised you, and why?
- Where would big O matter in a real app you use?

Check yourself

- One habit I will keep as a programmer

Try it this week

Take one program you wrote this year. Find its big O, replace a slow part with the right data structure, and protect it with five asserts, including edge cases.

Quiz sheet

Algorithms and software engineering: unit check

Programming · Level 4 · Unit check

Name _____ Class _____ Date _____

1 **An O(n²) algorithm takes 1 second for 1,000 items. About how long for 2,000 items?**

◯ About 1 second ◯ About 4 seconds ◯ About 2 seconds

2 **Binary search on a sorted list is O(log n).**

◯ True ◯ False

3 **Write has_pair(values, target): True if two different items add up to target. It must stay fast for 20,000 values, so use a set: for each v, is target − v already seen?**

```
def has_pair(values, target):
    # one pass with a set of the values seen so far
    return False
```

Write your program:

4 **fact(4) should be 24, but this returns 0. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  def fact(n):
2      if n == 1:
3          return 0
4      return n * fact(n - 1)
```

5 **Match each operation to what it costs or gives.**

x in a_list	• • • •	About one step
x in a_set	• • • •	Checks one by one: O(n)
stack.pop()	• • • •	The oldest item
queue.pop(0)	• • • •	The newest item

6 **Without memoisation, how many times does fib(5) work out fib(2)?**

◯ 1 ◯ 3 ◯ 5

7 **Put the lines in order: count the letters of “banana” with a dictionary and print how many a’s, 3. One line doesn’t belong.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn’t belong.

- [ ] `for c in "banana":`
- [ ] `count = {}`
- [ ] `print(count["a"])`
- [ ] `count[c] = 1`
- [ ] `count[c] = count.get(c, 0) + 1`

8 **Which are true about working test-first?**

- [ ] You can skip edge cases if the code looks right
- [ ] You tidy the code only while every test passes
- [ ] A new test should fail at first
- [ ] You write the test before the code

9 **Fill the gaps in the memoised Fibonacci.**

```
memo = {}
def fib(n):
    if n < 2:
        return n
    if n not in __________:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[__________]
```

Word bank: `n` `n - 1` `memo` `fib`

10 **Put the steps of binary search in order.**

- [ ] Otherwise keep the half that can hold the target
- [ ] Look at the middle item
- [ ] Repeat until the range is empty
- [ ] If it is the target, stop

11 **On a phone, a recursive function with no base case runs for ever.**

- ◯ True
- ◯ False

Check your answers

Algorithms and software engineering: unit check

Programming · Level 4 · Unit check

1 **About 4 seconds**

Double n, and n^2 work is multiplied by four: about 4 seconds.

2 **True**

Each look halves what is left, so the looks grow like log n: about 20 for a million items.

3

```
def has_pair(values, target):
    seen = set()
    for v in values:
        if target - v in seen:
            return True
        seen.add(v)
    return False
```

One pass, O(n): for each value, the partner it needs is target − v, and the set says in about one step whether it has been seen.

4 **Line 3: `return 1`**

The base case is reached, but with the wrong answer: everything is multiplied by 0.

5

x in a_list — Checks one by one: O(n)
x in a_set — About one step
stack.pop() — The newest item
queue.pop(0) — The oldest item

Lists search one by one; sets jump straight there; a stack gives the newest, a queue the oldest.

6 **3**

fib(5) calls fib(4) and fib(3); fib(2) appears under both, three times in all.

7 **Top to bottom: 2, 1, 4, X, 3**

```
count = {}
for c in "banana":
    count[c] = count.get(c, 0) + 1
print(count["a"])
```

get(c, 0) returns the count so far, so each letter adds one.

8 **You write the test before the code; A new test should fail at first; You tidy the code only while every test passes**

Red, green, refactor. Code that "looks right" is exactly where edge-case bugs hide.

9

```
memo = {}
def fib(n):
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

Work fib(n) out only if it isn't already in memo, then return the stored answer for n.

10 **Top to bottom: 3, 1, 4, 2**

The loop runs while the range lo…hi still has something in it: while lo <= hi.

11 **False**

The call stack fills up (at about 150 calls in MicroPython) and the program stops with an error.